

Step 4 — Structured Output and Streaming the Final Response — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (the real `.with_structured_output()` and `.stream()` calls), **Advanced** (why you need both from one shared answer, not two separate model calls). Read Basic first even if you've streamed a chain before — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — Two different shapes from one answer](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — Two different shapes from one answer

Basic Version

A person reading a support reply wants to watch words appear, like someone typing. Your logging and routing system wants one clean record it can check and sort on: what category, how confident, does a person need to look at this. Those are two different jobs, and LangChain has a separate tool for each — `.stream()` for the person, `.with_structured_output()` for the machine.

Things to use: - A Pydantic model, `TicketResolution`, for the structured record. - `model.with_structured_output(TicketResolution)` for the structured chain. - `.stream()` on the (already retry/fallback-hardened) drafting chain, for the human-readable version.

Intermediate Version

`.with_structured_output(SomeModel)` on a chat model returns a new runnable that, instead of a plain text reply, gives you back a real `SomeModel` instance — validated the moment it comes back, just like Doc05's structured-parser exercise, but as a first-class method on the model itself rather than a separate output-parser step.

`.stream(inputs)` on any `Runnable` returns an iterator (not a list) — each item you get from looping over it is one small piece of the final answer, as it's produced, not the whole thing computed up front and handed to you in pieces afterward. `for chunk in chain.stream(inputs): print(chunk, end="", flush=True)` is the real pattern — the `flush=True` matters, or your terminal may buffer the output and defeat the whole point of streaming it.

The exact pieces: - `from pydantic import BaseModel` for `TicketResolution`. - `resolution_prompt | model.with_structured_output(TicketResolution)` for the structured chain. - `for chunk in drafting_chain.stream(ticket_input): yield chunk` inside a generator function, so callers can consume it the same way regardless of what's underneath.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Here's the mistake this step is built to catch: calling the model twice — once to stream a human-readable draft, and a second, separate time to produce the structured record — and treating that as fine because "it's basically the same prompt." It isn't the same answer. Two separate model calls, even with nearly identical prompts, can genuinely disagree: the streamed draft might mention a refund timeline the structured summary doesn't capture, or the confidence score might not actually reflect what the streamed text says at all.

The correct shape: draft the answer once (Step 3's resilient chain), then treat that one piece of text as the shared source for both outputs — stream it as-is to the UI, and separately feed that same text into the structured-extraction chain to produce the `TicketResolution`. The structured step should be *summarizing and classifying what was already said*, not independently regenerating an answer that happens to resemble it.

Also decide, honestly: what should `needs_human_review` actually depend on? At minimum it should be `True` when Step 1's classifier flagged real ambiguity, or when Step 3's fallback fired — both of those are real signals you already have sitting around from earlier steps. Losing either one here, after building it earlier, would be the whole pipeline quietly throwing away information it worked to produce.

Difference between Basic, Intermediate, and Advanced: Basic explains why a person and a machine want different shapes from the same answer. Intermediate shows the real `.with_structured_output()` and `.stream()` calls. Advanced is the one design mistake that actually breaks this step even when both calls "work" individually: generating the human-readable and machine-readable versions from two separate model calls instead of one shared draft.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
draft the answer once, using Step 3's resilient/fallback drafting chain

to show a person: stream that exact draft, piece by piece

to log/route: feed that exact same draft text into a second chain that
extracts category, summary, confidence, and needs_human_review
```

Intermediate Version

```
# models.py -- add the second Pydantic model
from pydantic import BaseModel
```

```
class TicketResolution(BaseModel):
    category: str
    resolution_summary: str
    confidence: float
    needs_human_review: bool
```

```
# resolution.py
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from models import TicketResolution

RESOLUTION_PROMPT = ChatPromptTemplate.from_template(
    "Given this support ticket and the drafted reply, produce a resolution "
    "record.\n\nTicket: {ticket}\nDrafted reply: {draft}"
)

def build_resolution_chain():
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    return RESOLUTION_PROMPT | model.with_structured_output(TicketResolution)
```

Now write `stream_draft(drafting_chain, ticket_input)` yourself as a generator function, and wire `main.py` to draft once, stream that draft, then pass the same text into `build_resolution_chain()`, before checking the Advanced version below.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```
# resolution.py -- streaming helper, from the SAME chain the structured step read
def stream_draft(drafting_chain, ticket_input):
    full_draft = ""
    for chunk in drafting_chain.stream(ticket_input):
        full_draft += chunk
    yield chunk

# after streaming finishes, full_draft holds the exact text that was
# shown to the person -- this is what should be handed to the
# resolution chain next, not a second, freshly-generated draft.
```

```
# main.py -- one draft, two outputs
from handlers import billing_chain
from resolution import build_resolution_chain, stream_draft

ticket_input = {"ticket": "I was charged twice for my Pro subscription, order INV"}

print("Streaming draft to the UI:")
full_draft = ""
for piece in stream_draft(billing_chain, ticket_input):
    print(piece, end="", flush=True)
    full_draft += piece
print()

resolution_chain = build_resolution_chain()
resolution = resolution_chain.invoke({
    "ticket": ticket_input["ticket"],
    "draft": full_draft,
})
print(resolution)
```

Fill in the `needs_human_review` logic yourself (carrying forward Step 1's ambiguity flag and Step 3's fallback-fired flag), and compare your finished version against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic states the plain rule: draft once, use it twice. Intermediate shows the real `TicketResolution` model and `.with_structured_output()` chain. Advanced shows the actual streaming helper that captures the full draft text *while* streaming it, so the exact same content — not a re-generated approximation of it — feeds the structured step next.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)